University of Engineering & Technology Peshawar
(Nowshera campus)

Computer Programming

EE-170L

SUBMIDED BY

NAME	MUHAMMAD SULEMAN
REGISTRATION ID	BF25NWELEL0759
DEPARTMENT	ELECTRICAL ENGINEERING (POWER)
SECTION	A
LAB COPY	1ST TO 5TH
SEMESTER	2ND
SESSION	2025

LAB COPY SUBMITED TO

Engr. MUHAMMAD RIZWAN

SUBMISSION DATE 21/04/2026

1. Objectives

The primary goal of this lab session is to master the implementation of various iterative structures in C++. Loops are an essential programming mechanism that allow for the efficient repetition of tasks, eliminating the need for redundant code.

By the end of this lab, I aim to:

Write programs that utilize loops to generate specific numerical sequences.

Understand the fundamental logical differences between entry-controlled (while) and exit-controlled (do…while) loops.

Apply loop structures to solve mathematical problems, such as cumulative summation.

Implement nested loops to produce complex visual patterns or grids.

2. Theory

2.1 While Loop

The while loop is a control structure that executes a block of code as long as a specified Boolean condition remains true. Because the condition is evaluated at the beginning of the cycle, the loop body may never execute if the condition is initially false. It requires proper initialization, a clear termination condition, and a variable update to avoid infinite execution.

2.2 Nested While Loop

A nested loop consists of a loop placed inside the body of another loop. In this configuration, the outer loop typically manages the number of rows or major cycles, while the inner loop handles the individual repetitions (like columns) for every single iteration of the outer loop. This is a standard method for creating tables or geometric patterns.

2.3 Do...While Loop

The do...while loop functions similarly to the while loop, but with one critical distinction: it is an exit-controlled loop. This means the code block executes at least once before the condition is ever checked. It is particularly useful for scenarios requiring at least one action, such as requesting user input or passwords.

3. Programs and Outputs

Task 1: Print numbers 1 to 10 using a while loop

```
#include <iostream>

using namespace std;
```

```
int main () {
   // Initialize the counter variable at 1
   int i = 1;

   // Execute the loop as long as i is 10 or less
   while (i <= 10) {
      // Print the current value followed by a new line
      cout << i << endl;

      // Increment the counter to eventually end the loop
      i++;
   }
   return 0;
}
```

Output: 1 to 10 displayed in a single column.

Task 2: Repeat Task 1 using a do...while loop

```
#include <iostream>
using namespace std;

int main () {
   // Start the counter at 1
   int i = 1;

   // Enter the loop block immediately
   do {
```

```
        // Display the current value of i
        cout << i << endl;

        // Update the loop variable
        i++;
    } while (i <= 10); // Check if i is still within the range after the first run

    return 0;
}
```

Output: 1 to 10 displayed in a single column.

Task 3: Sum of numbers from 1 to n using a while loop

```
#include <iostream>
using namespace std;

int main () {
    // n stores the limit, sum stores the total, i is the counter
    int n, sum = 0, i = 1;

    // Get the target number from the user
    cout << "Enter a positive integer: ";
    cin >> n;

    // Accumulate the sum until i reaches n
    while (i <= n) {
        // Add the current value of i to the sum
        sum += i;
```

```
        // Move to the next integer
        i++;
    }

    // Output the final result
    cout << "Sum = " << sum << endl;
    return 0;
}
```

Output: Enter a positive integer: 5 | Sum = 15

4. Discussion

This lab demonstrated how iteration simplifies complex programming tasks. Instead of writing multiple lines of repetitive code, loops allow us to handle large-scale calculations or sequence generations efficiently. I observed that the while loop is best suited for logic where the number of runs is unknown but depends on a condition, whereas the do...while loop ensures an initial action is always taken. Mastering nested loops expanded my ability to create patterns and handle multi-dimensional data. Finally, the lab reinforced the importance of correctly updating loop variables to prevent the system from entering an infinite loop.

5. Conclusion

Through this lab session, I successfully applied while, do…while, and nested loop structures in C++. I have gained a clear understanding of the functional differences between pre-test and post-test loops. These exercises have improved my technical proficiency in controlling program flow and automating repetitive logic, which are vital skills for advanced software development